Transaction Webhook Service

Assignment Report | Design, implementation and verification

Live demonstration • Public API • GitHub source

1. What I set out to build

I built this service to receive transaction webhooks, acknowledge them quickly, and finish the work in the background. The main requirements were straightforward: accept a transaction, avoid creating it twice, keep its status in a database, simulate a 30-second processing step, and let a client check the result.

I also added a small browser interface. It gives someone reviewing the assignment a simple way to submit a transaction and look up its status without having to send requests manually through curl or Swagger.

2. How the service works

The application is written in Python with FastAPI. Pydantic validates incoming data, SQLAlchemy handles database access, and PostgreSQL stores the transaction records. Docker Compose provides a repeatable way to run the API and database together.

When a webhook arrives, the API validates the payload and attempts to insert it into PostgreSQL. The database enforces uniqueness on transaction_id. If the insert succeeds, the API returns HTTP 202 and schedules the simulated processing task. The task waits for the configured delay (30 seconds by default), then updates the record to PROCESSED. The status endpoint reads the saved record, and the browser UI uses the same API.

One implementation detail is worth being clear about: the background task currently runs inside the FastAPI process. The recovery loop checks for records still marked PROCESSING and schedules them again. This is adequate for the assignment demonstration, but it is not the same as operating a separate worker service.

Technology at a glance

Part	What I used	Why it is there
API	FastAPI	Defines the endpoints and returns the webhook acknowledgement.
Validation	Pydantic	Checks required fields, amount and currency before storage.
Database	PostgreSQL + SQLAlchemy	Stores transaction details, status and timestamps.
Background work	asyncio tasks in the API process	Simulates the delay and updates the transaction status.
Packaging	Docker + Docker Compose	Makes the local setup repeatable and supports deployment.
Browser UI	Compiled frontend served by FastAPI	Lets a reviewer submit transactions and check their status.

3. API available for testing

GET / returns a health response with the current UTC time.

POST /v1/webhooks/transactions accepts transaction_id, source_account, destination_account, amount and currency. A valid request is stored before the service responds with HTTP 202 and {"status":"accepted"}. The response header X-Webhook-Result indicates whether the record was newly created or was a duplicate.

GET /v1/transactions/{transaction_id} returns the stored transaction as a one-element JSON array. The status is PROCESSING while the simulated work is underway and PROCESSED when it finishes. processed_at remains null until completion. An unknown ID returns 404.

GET /ui/ serves the browser interface when the compiled frontend is included in the API image. The interactive API documentation is available at /docs.

4. Why I chose this design

FastAPI keeps the API small and readable. It gives me request validation, clear endpoint definitions and generated OpenAPI documentation without adding much boilerplate.

PostgreSQL is the source of truth. I write the transaction before acknowledging the webhook, so the request is not reported as accepted unless the record has been stored. It also gives the status endpoint a reliable place to read from.

The unique transaction_id constraint is the key part of duplicate protection. I used INSERT … ON CONFLICT DO NOTHING so the database, rather than an in-memory check, decides whether a transaction is new. This also handles concurrent attempts to insert the same ID.

I return HTTP 202 because the transaction has been accepted, not completed. The processing step happens later, so a success response such as 200 would give the wrong impression about its state.

For the amount, I use Decimal in Python and NUMERIC(18,2) in PostgreSQL. This avoids relying on binary floating-point values for money.

I used PostgreSQL for both persistence and the simple work queue instead of introducing Redis, Celery or Kafka. For the scale of this assignment, fewer moving parts make the solution easier to run and explain. The trade-off is that a production system with multiple worker instances would need stronger job-claiming and coordination.

Docker Compose makes local setup repeatable. For the public demonstration, I deployed the API and database on one Oracle Cloud Ubuntu VM. PostgreSQL is intended to remain private; the API is exposed on port 8000.

5. How I checked the requirements

I treated each requirement as something that should be observable and testable, rather than relying only on the code looking correct. The table below maps the assignment expectations to the implementation and the check I would use.

Assignment expectation	Implementation	How to verify
Accept a transaction webhook	POST /v1/webhooks/transactions	Send the sample JSON; expect HTTP 202.
Respond within 500 ms	Validate and insert before returning; delay runs in background	Measure POST latency. Current automated test checks one request under 0.5 s; run load testing for stronger evidence.
Simulate 30-second processing	PROCESSING_DELAY_SECONDS defaults to 30	Read status immediately, then again after about 30 seconds.
Avoid duplicate records	Unique transaction_id + INSERT ON CONFLICT DO NOTHING	Submit the same ID repeatedly and concurrently; verify only one row exists.
Persist transaction status	PostgreSQL transaction table	Restart API without deleting the database volume and query the record.
Retrieve transaction status	GET /v1/transactions/{transaction_id}	Check fields and status; unknown IDs should return 404.
Health check	GET /	Expect HTTP 200, HEALTHY and a UTC timestamp.
Reject invalid input	Pydantic validation	Send missing or invalid fields; expect HTTP 422.
Handle database failure	SQLAlchemy exception handling	Stop PostgreSQL; webhook should return 503 rather than 202.
Provide a public demo	Oracle Cloud VM, Docker Compose, port 8000	Call the service from outside the VM.
Provide a browser interface	Frontend served at /ui/	Submit through the page and confirm it reads

		status from the API.

6. Verification strategy

The existing automated tests cover the health route, the webhook response, the initial PROCESSING state, eventual PROCESSED state, duplicate submissions, concurrent duplicate inserts, invalid payloads and unknown transaction IDs.

For the 500 ms requirement, I measure the POST request itself. The current test asserts that one request completes in under 0.5 seconds. To make a stronger performance claim, I would run a repeatable load test and report the request count, concurrency, environment, p50, p95, p99, maximum latency and error rate. One fast request is useful, but it is not enough to prove performance under load.

I would also test the deployed service from a machine outside the VM: check the health route, submit a new transaction, read it immediately, wait for the processing delay, and read it again. Finally, I would restart the API while a transaction is still PROCESSING and confirm that the recovery loop picks it up.

Practical test checklist

Test	What I do	What should happen
Normal transaction	Submit a new transaction and check it immediately and after the delay.	202; PROCESSING with null processed_at; then PROCESSED with a timestamp.
Duplicate delivery	Send the same transaction ID multiple times, including concurrent requests.	Requests are acknowledged, but only one database record is created.
Invalid payload	Leave out required fields or use an invalid amount/currency.	422 response and no transaction record.
Unknown transaction	Look up an ID that was never submitted.	404 response.
Database unavailable	Stop PostgreSQL and submit a webhook.	503 response; the API must not claim the event was stored.
Persistence	Submit a transaction, restart the API and retain the database volume.	The transaction can still be retrieved.
Recovery	Restart the API while a transaction is PROCESSING.	The recovery loop schedules it again and it eventually completes.
Latency	Measure many POST requests at a stated concurrency.	Report p50/p95/p99/max and error rate; assess against the 500 ms target.
Browser flow	Submit a transaction through /ui/ and watch its status.	The page shows acceptance and updates to PROCESSED.
External smoke test	Call health, submit and status endpoints from another network.	All public routes respond; PostgreSQL is not exposed publicly.

7. Current limitations and next improvements

The current background scheduler lives in the API process. Before running multiple API instances or treating this as a production service, I would move processing into a separate worker and claim jobs atomically in PostgreSQL, for example with SELECT … FOR UPDATE SKIP LOCKED.

The recovery loop reloads PROCESSING records and schedules them in memory. It can resume work after an API restart, but it does not use leases, claim tokens or durable retry metadata. A restart can therefore cause the simulated 30-second delay to start again.

If the same transaction ID arrives with different account details or a different amount, the current code keeps the first record and treats the later request as a duplicate. A stricter API contract could compare the payload and return 409 Conflict when the contents do not match.

The status model currently has PROCESSING and PROCESSED. A more complete implementation would add QUEUED and FAILED states, retry counts, backoff and a clear policy for transactions that cannot be completed.

The public demo currently uses HTTP and a raw IP address. Before handling real transaction or personal data, I would configure HTTPS, restrict inbound access, and add the authentication or webhook-signature checks required by the payment provider. I would also add a readiness check that confirms the database is reachable; the current root route only confirms that the API responds.

8. Running and reviewing the project

The GitHub repository contains the FastAPI application, database and worker modules, schemas, Docker configuration, tests, README and frontend build integration. For local development, the documented Docker Compose command starts the API and database. The UI is served at /ui/ and the API documentation is served at /docs.

Live UI: http://130.210.26.235:8000/ui/

API documentation: http://130.210.26.235:8000/docs

Health check: http://130.210.26.235:8000/

Source code: https://github.com/Muskansuman/Transaction-Webhook-Service

9. Closing note

The result is a working demonstration of the core webhook flow: validate the event, store it before acknowledging, prevent duplicate records at the database level, process it asynchronously, and make the final status available through an API and a small browser interface.